

Being a Team Member

Understanding Jira

What is Agile?

The Foundation of Modern Software

Agile breaks down large projects into small, manageable chunks delivered in short cycles called sprints (typically 2-4 weeks):

- **Iterative delivery:** Ship working software regularly
- **Customer collaboration:** Gather feedback continuously
- **Respond to change:** Adapt plans based on real-world learning
- **Self-organising teams:** Trust people closest to the work

Why Agile?

Traditional waterfall methods plan everything upfront. Agile acknowledges that requirements change, and builds flexibility into the process.

Understanding Scrum

The Most Popular Agile Framework

Scrum provides structure to Agile principles through defined roles, events, and artefacts.

Key Roles:

- **Product Owner:** Defines what to build (the "what" and "why")
- **Scrum Master:** Removes blockers, facilitates process
- **Development Team:** Builds the product

Key Events:

- **Sprint Planning:** Commit to work for the sprint
- **Daily Stand-up:** 15-minute sync on progress
- **Sprint Review:** Demonstrate completed work
- **Sprint Retrospective:** Improve team processes

Epics - The Big Picture

Grouping Related Work

An **Epic** represents a large body of work that cannot complete in a single sprint. Think of it as a major feature or initiative.

Characteristics:

- Too large to complete in one sprint
- Broken down into smaller User Stories
- Tracked over multiple sprints
- Has a clear business objective

Example:

- **Epic:** "Implement Single Sign-On (SSO) across all applications"

This might take **3-6 sprints** and include stories about authentication, user migration, testing, and documentation.

User Stories - The Building Blocks

Describing Features from the User's Perspective

A **User Story** describes functionality from an end-user viewpoint. It's a promise for a conversation, not a complete specification.

Standard Format:

"As a [type of user], I want [goal], so that [reason/benefit]"

Example:

"As a customer, I want to reset my password via email, so that I can regain access without contacting support."

Good User Stories are INVEST:

- **Independent:** Can be developed separately
- **Negotiable:** Details emerge through discussion
- **Valuable:** Delivers user benefit
- **Estimable:** Team can size the effort
- **Small:** Fits within a sprint
- **Testable:** Clear success criteria exist

Tasks - Breaking Down the Work

The Technical Implementation Details

Tasks represent the technical steps needed to complete a User Story. They're created by the development team during sprint planning.

Characteristics:

- Specific, actionable work items
- Assigned to individual team members
- Typically completed in hours or days
- Technical in nature

Example User Story:

"As a customer, I want to reset my password via email"

Tasks might include:

- Create password reset API endpoint
- Design email template
- Implement token generation and validation
- Write unit tests
- Update user documentation

Bugs - Tracking Defects

Managing Issues in Production and Development

A **Bug** represents incorrect behaviour in existing functionality. Unlike User Stories, Bugs describe problems, not new features.

When to Create a Bug:

- Existing functionality doesn't work as designed
- Error conditions occur unexpectedly
- Performance degrades below acceptable levels

Bug Priority Considerations:

- **Critical:** Production down, data loss risk
- **High:** Major feature broken, workaround difficult
- **Medium:** Feature partially broken, workaround exists
- **Low:** Cosmetic issues, minor inconveniences

Example:

"Password reset emails fail for users with plus signs (+) in email addresses"

Jira Ticket Structure

Standardising How We Document Work

A consistent ticket structure ensures clarity and completes handoffs between teams.

Background:

As a [Product/Service Owner], I want a [Data Analyst/Data Engineer/Platform Engineer] to [specific action], so that [business outcome].

Environments:

- Development
- UAT/Preproduction
- Production

Changes:

- Detailed description of what will change
- Infrastructure modifications
- Configuration updates
- Deployment steps

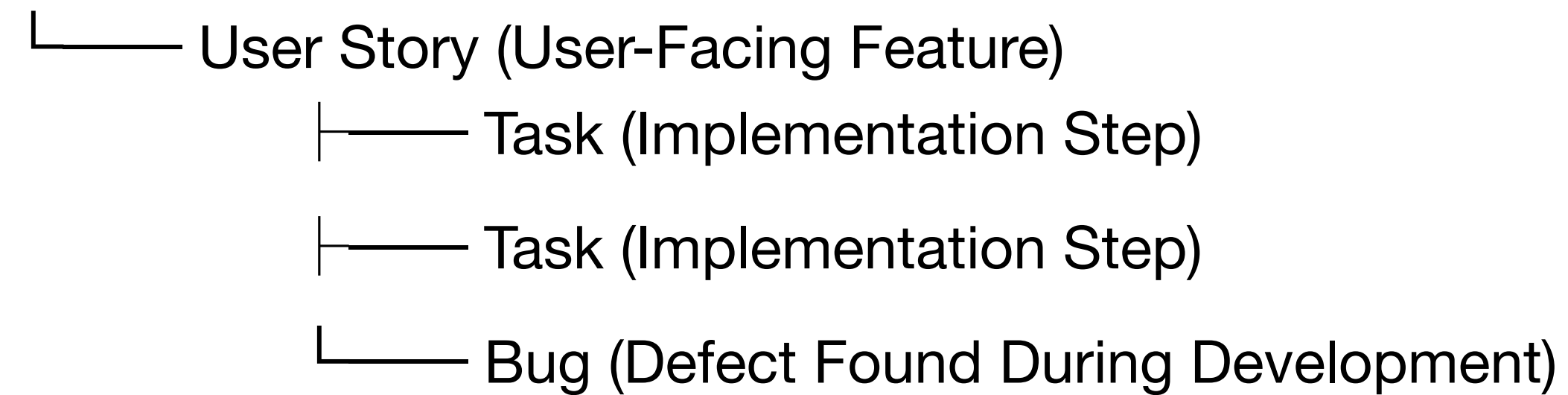
Acceptance Criteria:

- Specific, testable condition 1
- Specific, testable condition 2

Putting It All Together

The Hierarchy and Workflow

Epic (Strategic Initiative)



Typical Workflow:

- Product Owner creates Epic for strategic goal
- Epic breaks into User Stories in backlog refinement
- Team selects Stories for sprint during planning
- Team creates Tasks to implement Stories
- Bugs emerge from testing or production
- Team addresses Bugs based on priority

End of the training...

See you around, **[student name]**.

Jira structures work because they match how teams naturally think —from strategic goals down to concrete tasks—creating shared understanding without imposing rigid bureaucracy. Master these fundamentals, then adapt them to your context, because the goal isn't perfect tickets but shipped software that solves real problems.

Thank you, [student name].

Any Questions?